CipherComply: Final Implementation Specification (v5 — FROZEN)

Status: Frozen for implementation. This is the last document-level revision. All findings from five review rounds are resolved, pinned, or explicitly gated to the validation spike. Remaining open items are **compile-dependent** or **process** items that no further document revision can close — they are listed in section 15 and must be closed against the live toolchain, not on paper. No Compact code here has been compiled and no independent audit has been run; this is a specification, not a production security attestation.

Date: 2026-10-09 **Supersedes:** v4, v3, v2, and the original "audited build" spec. **Freeze rationale:** across rounds, finding severity fell from foundational breaks (unbound digest) to correctness hardening (float money, idempotency binding). The latest audit's own verdict matches this document's (develop now; Preprod only after compiling and proving; Mainnet not yet). Further paper review has diminishing returns; the gate is now execution.

---

0. Changes since v4 (this round)

Resolved from the v4 audit:

- **C2 — authenticated identity bound to subject before signing (critical).** v4 trusted a client-supplied `subjectHex`. v5 requires the gateway to **verify** that the authenticated principal controls the secret behind the subject (wallet-signature proof over a server challenge) **before** the oracle signs. The oracle never signs a subject the caller has not cryptographically proven ownership of. See 5.6 and 7.3.
- **H1 — monotonic, bounded revocation.** `revokeSubject` now enforces `newRevokedAt >= existingRevokedAt` and is tied to chain time once the time primitive lands.
- **H2 — crash-safe registry.** `reserve → issue → confirm` replaced by a durable `RESERVED → ISSUED → CONFIRMED` state machine; an `ISSUED` row never silently expires. See 7.1.
- **H3 — idempotency bound to the request.** `Idempotency-Key` is keyed to `principal + endpoint + canonical-request-hash`; reuse with a different body is a `422`, not a stale replay.
- **H4 — money is no longer floating-point.** Amounts are scaled integers (minor units) with explicit currency precision and rounding rules.
- **H5 — Schnorr signer pinned where possible.** Key generation, encoding, custody, rotation, and golden vectors specified; only the exact in-circuit verifier call stays spike-dependent.
- **M1** — `verificationMethod` explicitly classified as non-authoritative metadata (the authoritative tier is the signed `assuranceLevel`).
- **M2** — "independent extraction" defined: a different model/provider, not the same prompt twice.
- **M3** — exact version pins with checksums instead of `compact update 0.31`.
- **C3** — compiler-safety claim now cited inline to authoritative Midnight sources (section 15), not asserted.

---

1. Product summary and trust model

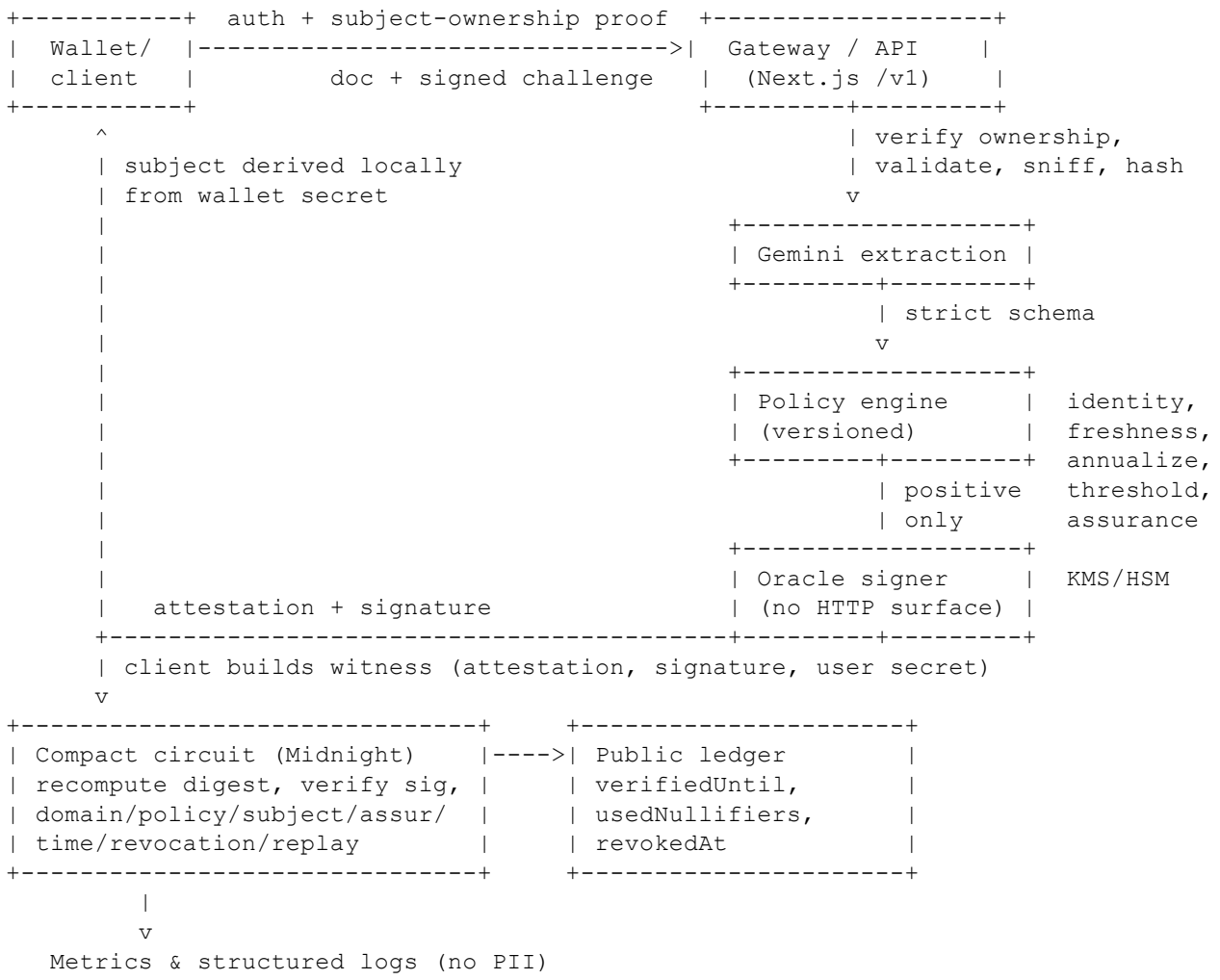
Privacy-preserving eligibility attestation for Web3 protocols. Sensitive evidence stays off the public ledger; only a narrowly scoped result is recorded on Midnight.

- The **oracle** (document pipeline + policy engine + signing key) decides eligibility. The ZK circuit proves **only** that a valid, fresh, unused attestation from the authorized oracle exists for the caller's subject, policy, and domain.
- ZK does **not** prove the document is genuine, belongs to the subject, or was read correctly by the AI model. This Garbage-In/Garbage-Out boundary is the system's **primary residual risk** and remains an explicit trust assumption;

hardening the circuit does not reduce it.
- The oracle is a **privacy-trusted party**; it sees plaintext and can link subjects to documents.
- Raw evidence is off-ledger. In server-mediated mode it is **not** off the server or AI provider; only TEE or on-device modes remove that exposure.

---

2. Architecture (C4 container level)



---

3. Data classification

Data	Public on-chain?	Handling
Raw document, income, tax figures	No	Off-chain; retention policy required
Attestation, signature, document hash	No	Private witness; optional audited off-chain retention
Oracle public key, policy ID, domain, min-assurance	Yes	Trust anchors
Subject (derived from user secret + contract)	Yes	Pseudonymous, persistent
statusValidUntil	Yes	Downstream expiry checks
Used-attestation nullifier	Yes	Replay protection
revokedAt[subject]	Yes	Revocation epoch

Every `ledger` field is public; store commitments/hashes, never raw personal data, never a plain hash of a guessable value.

---

4. Threat model and invariants

Threat	Mitigation
Payload tampering	Digest recomputed in-circuit; signature over that digest
Cross-user substitution (redemption)	Subject derived in-circuit from caller's secret vs signed subject
Cross-user substitution (signing)	Gateway verifies caller owns the subject before the oracle signs (5.6, 7.3)
Cross-contract/network replay	`domain` bound; subject derivation includes `kernel.self()`
Replay (same attestation twice)	Nullifier from nonce, consumed on-chain
Stale attestation (expiry)	`expiresAt` vs chain time — **[SPIKE-BLOCKER]** (4.1)
Stale status	`statusValidUntil`; revocation circuit
Revoked-then-resurrected	Reject `issuedAt < revokedAt[subject]`
Inconsistent revocation epoch	`revokeSubject` monotonic + chain-time bound
Backward status overwrite	Monotonic `verifiedUntil` guard
Forged assurance tier	`assuranceLevel` signed inside the attestation
Document re-earn after crash	Durable registry state machine (7.1)
Idempotency confusion	Key bound to principal + endpoint + request hash
Float threshold/rounding error	Scaled-integer money (7, H4)
Wrong policy	`policyId` signed and asserted
Rogue/compromised oracle	Mutable key, admin-gated rotation, key ID, KMS/HSM
Open-oracle / collusion	Threshold/multisig signer, audit logging, multi-source cross-check — not in-circuit
Forged/borrowed document	Holder-name match, issuer allow-list, document uniqueness, verifiable sources, review
Hallucination/prompt injection	Output bounds, independent dual extraction, evidence refs, deterministic policy, ambiguity to review
Under-constrained circuit	Constrain every witness value; adversarial tests + external review
Endpoint abuse / DoS	Auth, rate limits, quotas, idempotency, timeouts
Fake UI confirmation	Only finalized, indexed Midnight tx drives "confirmed"

4.1 Expiry enforcement — the one remaining release blocker

Freshness requires rejecting attestations past `expiresAt` using **chain-authoritative time**, never a client timestamp. The exact ledger-8 primitive for reading block time in a Compact circuit was not confirmable from the public docs reviewed. **Profile A must not deploy with this assertion commented out.** The spike must confirm the primitive and activate the assertion; CI fails the build if it is absent.

4.2 Circuit invariants (all must hold)

1. Signed digest equals the hash of the in-circuit struct (all fields, incl. `assuranceLevel`).
2. Signature verifies under the current oracle key.
3. `domain` and `policyId` equal ledger values.
4. Signed subject equals `subjectOf(callerSecret)`.
5. `assuranceLevel >= minAssurance`.
6. Chain time is before `expiresAt` **[SPIKE-BLOCKER]**.
7. `issuedAt >= revokedAt[subject]` (or no revocation on record).
8. Nullifier not previously consumed; then consume it.
9. New `statusValidUntil` not earlier than any stored value (monotonic).

10. `eligible` is true.
11. Only subject, `statusValidUntil`, nullifier, revokedAt become public.

---

5. Contract profiles

Profile	Toolchain	Lang	Ledger	Signature	Use
A — Public network (PRIMARY)	0.31.1	0.23	8	`jubjubSchnorrVerify`	Deployable today
B — Ed25519 reference	0.35.x `--feature-zkir-v3`	0.27	9	`ed25519Verify<#n>`	Forward-looking

Verified against the Midnight compatibility matrix: all public environments run ledger 8 on compiler 0.31.1 / compact-runtime 0.16.0 / on-chain runtime 3.0.0 / Midnight.js 4.1.1 / proof server 8.1.0. 0.34/0.35 target ledger 9 and cannot deploy publicly.

5.1 Shared attestation and derivations

```
struct Attestation {
  schemaVersion: Uint<8>;
  domain: Bytes<32>;
  policyId: Bytes<32>;
  subject: Bytes<32>;
  eligible: Boolean;
  assuranceLevel: Uint<8>;        // 1=low 2=medium 3=high — SIGNED
  amountMinorUnits: Uint<64>;     // scaled integer, not float
  currency: Bytes<3>;             // ISO 4217
  issuedAt: Uint<64>;
  expiresAt: Uint<64>;
  statusValidUntil: Uint<64>;
  nonce: Bytes<32>;
  documentHash: Bytes<32>;        // bound into signature, never on-chain
}

circuit subjectOf(secretKey: Bytes<32>): Bytes<32> {
  return persistentHash<Vector<3, Bytes<32>>>(
    [pad(32, "cc:subject:v1"), kernel.self().bytes, secretKey]);
}
circuit nullifierOf(nonce: Bytes<32>): Bytes<32> {
  return persistentHash<Vector<3, Bytes<32>>>(
    [pad(32, "cc:nullifier:v1"), kernel.self().bytes, nonce]);
}
circuit adminCheck(sk: Bytes<32>): Boolean {
  return adminHash == persistentHash<Vector<2, Bytes<32>>>(
    [pad(32, "cc:admin:v1"), sk]);
}
```

The digest MUST be an exported pure circuit so the signer runs identical code.

5.2 Profile A — public network (Schnorr, ledger 8) — PRIMARY

```
pragma language_version 0.23;
import CompactStandardLibrary;

export sealed ledger domainTag: Bytes<32>;
export sealed ledger policyId: Bytes<32>;
export sealed ledger minAssurance: Uint<8>;
export ledger oraclePublicKey: Bytes<32>;
export ledger oracleKeyId: Uint<32>;
```

```
export ledger adminHash: Bytes<32>;
export ledger verifiedUntil: Map<Bytes<32>, Uint<64>>;
export ledger revokedAt: Map<Bytes<32>, Uint<64>>;
export ledger usedNullifiers: Map<Bytes<32>, Boolean>;

witness userSecretKey(): Bytes<32>;
witness adminSecretKey(): Bytes<32>;
witness attestation(): Attestation;
witness oracleSignature(): Bytes<64>;

constructor(domain_: Bytes<32>, policy_: Bytes<32>, minAssurance_: Uint<8>,
            oracleKey_: Bytes<32>, adminHash_: Bytes<32>) {
  domainTag = disclose(domain_);
  policyId = disclose(policy_);
  minAssurance = disclose(minAssurance_);
  oraclePublicKey = disclose(oracleKey_);
  oracleKeyId = 1;
  adminHash = disclose(adminHash_);
}

export pure circuit attestationDigest(a: Attestation): Bytes<32> {
  return persistentHash<Attestation>(a);
}

export circuit verifyAndRegister(): [] {
  const a = attestation();
  const digest = attestationDigest(a);

  assert(a.schemaVersion == 1, "Unsupported schema");
  assert(a.domain == domainTag, "Wrong domain");
  assert(a.policyId == policyId, "Wrong policy");
  assert(a.subject == subjectOf(userSecretKey()), "Subject mismatch");
  assert(a.assuranceLevel >= minAssurance, "Assurance below policy minimum");
  assert(a.issuedAt < a.expiresAt, "Invalid window");
  assert(a.expiresAt <= a.statusValidUntil, "Invalid status window");

  // [SPIKE-BLOCKER] Freshness vs chain time. MUST be active before deploy.
  // assert(blockTimeLt(a.expiresAt), "Attestation expired");

  assert(jubjubSchnorrVerify(oraclePublicKey, digest, oracleSignature()),
         "Invalid oracle signature");

  if (revokedAt.member(a.subject)) {
    assert(a.issuedAt >= revokedAt.lookup(a.subject), "Issued before revocation");
  }

  const nf = nullifierOf(a.nonce);
  assert(!usedNullifiers.member(nf), "Attestation already used");
  assert(a.eligible, "Not eligible");

  if (verifiedUntil.member(a.subject)) {
    assert(a.statusValidUntil >= verifiedUntil.lookup(a.subject),
           "Status would move backward");
  }

  usedNullifiers.insert(disclose(nf), true);
  verifiedUntil.insert(disclose(a.subject), disclose(a.statusValidUntil));
}

export circuit rotateOracleKey(newKey: Bytes<32>): [] {
  assert(adminCheck(adminSecretKey()), "Not admin");
  oraclePublicKey = disclose(newKey);
  oracleKeyId = (oracleKeyId + 1) as Uint<32>;   // explicit cast, see 5.5
}
```

```
// Monotonic + chain-time-bound revocation (H1).
export circuit revokeSubject(subject: Bytes<32>, atTime: Uint<64>): [] {
  assert(adminCheck(adminSecretKey()), "Not admin");
  // [SPIKE-BLOCKER] once the time primitive is confirmed, also assert:
  // assert(atTime <= currentChainTime(), "Revocation in the future");
  if (revokedAt.member(subject)) {
    assert(atTime >= revokedAt.lookup(subject), "Revocation epoch moved backward");
  }
  revokedAt.insert(disclose(subject), disclose(atTime));
  verifiedUntil.remove(disclose(subject));
}
```

5.3 Profile B — Ed25519 (ledger 9, devnet)

Identical body with `pragma language_version 0.27;`, compiled `--feature-zkir-v3`, substituting `ed25519Verify<32>(...)`. Not deployable publicly until ledger 9 ships.

5.4 Design notes

Only `verifiedUntil` is stored (no `verifiedAt`). Key rotation invalidates unsubmitted attestations (safe default); add `keyId` to the struct + grace window to change that. Admin-by-secret is prototype-grade; production uses multisig/threshold governance with logged actions. `kernel.caller()` is `none()` for top-level user calls off-chain, so the user-secret derivation is the correct binding.

5.5 Verified Uint rules (compiler 0.31.1)

1. `Uint<32> + 1` widens to `Uint<0..4294967297>`; cast back: `(x + 1) as Uint<32>`.
2. Multi-party key registries must use deterministic keys; a sequence increment invalidates registered keys. The oracle key is stored and rotated explicitly (compliant).
3. CI rejects compiler 0.31.0 (silent range-constraint drop, undetectable by source review); requires >= 0.31.1.

5.6 Subject-ownership binding (C2)

The subject the oracle signs MUST be one the caller provably owns. Before signing, the gateway requires a wallet signature over a server challenge `H(domain, policyId, serverNonce, subjectHex)`. The server recomputes `subjectOf` from the proven key material (or verifies the signature against the public key that derives the subject) and rejects any mismatch. The oracle signs only after this check. This closes the gap where an authenticated user could request a signature for a subject they do not control.

---

6. Oracle signer

```
import { pureCircuits } from "../contracts/managed/ciphercomply/contract"; // [SPIKE path]

export interface OracleSigner {
  publicKey: Uint8Array;                          // 32 bytes
  sign(digest: Uint8Array): Promise<Uint8Array>;  // 64-byte signature
}

// Profile B prototype: Node native Ed25519.
import { sign as edSign, createPrivateKey } from "node:crypto";
export function localEd25519Signer(pkcs8Der: Buffer, publicKey: Uint8Array): OracleSigner {
  const key = createPrivateKey({ key: pkcs8Der, format: "der", type: "pkcs8" });
  return { publicKey, sign: async (d) => new Uint8Array(edSign(null, d, key)) };
}
```

```
export async function issueAttestation(signer: OracleSigner, a: Attestation) {
  const digest = pureCircuits.attestationDigest(a);   // identical to circuit
  const signature = await signer.sign(digest);
  return { attestation: a, signature, digest };
}
```

Profile A signer specification (H5): key is generated on the Jubjub embedded curve matching `jubjubSchnorrVerify`; private key held in KMS/HSM, never browser-visible; public key is the 32-byte encoding written to `oraclePublicKey`; signatures are the 64-byte encoding the verifier expects; rotation via on-chain `rotateOracleKey` with a documented runbook; golden vectors lock (message, pubkey, signature, digest) for the TS signer and the compiled verifier. **Spike-dependent only:** the exact Compact `jubjubSchnorrVerify` argument order/encoding and the matching Midnight signing primitive — do not hand-roll Schnorr. A reviewed `@noble/ed25519` (RFC 8032) is acceptable for Profile B Ed25519 only.

Rules: 32-byte CSPRNG nonces, never reused; **positive attestations only**; the oracle refuses to issue for a revoked subject; signer has no HTTP surface and is callable only by the policy engine.

---

7. Document pipeline and policy engine

Deployment modes: server-mediated (default; oracle + AI see plaintext), TEE-backed, on-device. E2E encryption with server-side AI is self-contradictory and not offered.

Ingestion: authenticated session + subject-ownership proof (5.6); per-subject and per-IP rate limits + monthly quota; magic-byte sniffing (not client `Content-Type`); size limit from the real hosting cap (base64 adds ~33%); Gemini Files API for large docs **[SPIKE: confirm current limits/MIME]**; SHA-256 of exact bytes is `documentHash`; no content in logs, no raw error text to clients.

Extraction schema (Gemini structured output):

```
documentType: enum[tax_return, payslip, bank_statement, employer_letter, other]
holderName: string
issuer: string
documentDate: ISO date
periodStart / periodEnd: ISO date
amountMinorUnits: integer    // scaled integer (H4), e.g. cents; no floats
currency: ISO 4217
amountPeriod: enum[annual, monthly, weekly, per_document]
evidence: [{ page: int, quote: string (<= 200 chars) }]
ambiguous: boolean
ambiguityReason: string
```

Money handling (H4): all amounts are integer minor units with explicit per-currency precision (e.g. 2 for USD, 0 for JPY). Annualization and threshold comparison are integer arithmetic with documented rounding (round half-up at defined boundaries). No IEEE-754 anywhere in the decision path.

Deterministic policy (versioned, in code):

1. Reject unless output validates strictly. Parse failures abort with **no signature**.
2. Allowed `documentType` + issuer/template allow-list per policy.
3. `holderName` matches the authenticated KYC identity (fuzzy match, documented threshold); the result sets `assuranceLevel`.
4. `documentDate` within freshness window.
5. Annualize by `amountPeriod` in integer minor units; non-base currency only under explicit FX policy.
6. Sanity bounds; implausible or `ambiguous` routes to manual review.

7. **High-value policies — independent cross-check (M2):** a second extraction via a *different model or provider* (not the same prompt re-run), requiring agreement within tolerance. Same-model re-runs do not count as independent.
8. **Document uniqueness** via the durable registry (7.1). The document hash does not go on-chain.
9. Threshold compare in integer units; on pass, issue a positive attestation carrying `assuranceLevel`, `amountMinorUnits`, `currency`; else return a reason code.

Preferred evidence sources: verifiable inputs (tax-authority/account-aggregator APIs, signed documents, verifiable credentials) over AI-read PDFs; PDF extraction is the lower-assurance fallback.

7.1 Document-uniqueness registry — durable state machine (H2)

States: `RESERVED → ISSUED → CONFIRMED`, persisted transactionally keyed by content fingerprint.

1. `reserve`: atomic insert `RESERVED(fingerprint, subject, ts)`. Conflict with a live row for a different subject → `409`. `RESERVED` may expire (abandoned uploads).
2. `markIssued`: transition to `ISSUED` in the **same transaction** that records the signed attestation's nonce/digest. **`ISSUED` never silently expires** — a crash after signing leaves a durable `ISSUED` row, so the document cannot be re-earned.
3. `confirm`: transition to `CONFIRMED` after on-chain settlement is observed.

Recovery: a process that finds an `ISSUED` row without on-chain confirmation re-drives settlement rather than re-signing. Idempotency (7.2) ties the original request to that `ISSUED` row.

7.2 Idempotency (H3)

`Idempotency-Key` is bound to `authenticatedPrincipal + endpoint + canonicalRequestHash`. Same key + same body → original result replayed. Same key + **different** body → `422 idempotency_key_reuse`. Keys are stored with the response for a documented TTL.

7.3 Gateway route outline

```
export async function POST(req: NextRequest) {
  const session = await requireSession(req);                    // 401
  await verifySubjectOwnership(session, req);                   // 403 if proof fails (5.6)
  const idem = req.headers.get("Idempotency-Key");              // 400 if missing
  const reqHash = canonicalRequestHash(session.principal, "/v1/verify", req);
  const cached = await idempotencyLookup(idem, reqHash);        // 422 on key+body mismatch
  if (cached) return json(cached.status, cached.body);

  await enforceRateLimit(session.subjectId);                    // 429
  const { bytes, subject } = await parseAndValidate(req);       // 400/413/415
  const fingerprint = contentFingerprint(bytes);
  const documentHash = sha256(bytes);

  const extracted = await extract(bytes);                       // strict schema or throw
  const verdict = evaluatePolicy(extracted, session.identity, POLICY_V1);
  if (verdict.status !== "eligible")
    return finalize(idem, reqHash, 422, { status: verdict.status, reason: verdict.reasonCode });

  await reserveDocument(fingerprint, subject);                  // 409 on conflict
  const nowS = Math.floor(Date.now() / 1000);
  const issued = await issueAttestation(signer, {
    schemaVersion: 1, domain: DOMAIN, policyId: POLICY_ID, subject,
    eligible: true, assuranceLevel: verdict.assuranceLevel,
    amountMinorUnits: verdict.amountMinorUnits, currency: verdict.currency,
    issuedAt: nowS, expiresAt: nowS + 900,
    statusValidUntil: nowS + STATUS_TTL, nonce: randomBytes(32), documentHash,
  });
```

```
  await markIssued(fingerprint, subject, issued);              // durable, same tx
  return finalize(idem, reqHash, 201, { status: "eligible",
    verificationMethod: verdict.method,   // non-authoritative metadata (M1)
    ...serialize(issued) });
}
// catch: log internally with a correlation id; return { error: "verification_failed" }
```

---

8. API contract

Versioned `/v1`; JSON; errors `{ error, reason?, correlationId }`.

POST /v1/verify — headers `Authorization`, subject-ownership proof, `Idempotency-Key` (required).

Status	Meaning
201	Eligible; attestation returned
400	Malformed / missing field / missing Idempotency-Key
401	Missing/invalid session
403	Subject-ownership proof failed
409	Document claimed by another subject
413	Over size limit
415	Failed magic-byte / MIME check
422	Ineligible, expired, or idempotency key+body mismatch
429	Rate limit / quota
500	Technical failure (generic body)

201 body: `status`, `verificationMethod {method, assurance}` (**non-authoritative; the signed `assuranceLevel` is authoritative, M1**), and `attestation` (all struct fields + `signatureHex`).

GET /v1/status/ — safe, idempotent; `{ verified, validUntil }` or `404`.

---

9. Identity model and assurance tiers

Subject = `subjectOf(userSecret)`; deterministic, contract-scoped, pseudonymous, unforgeable without the secret, and **ownership-verified before signing** (5.6).

method	assuranceLevel	Basis
emailOtp	1 (low)	Email OTP
walletSignature	2 (medium)	Wallet key control
govIdCamera / KYC	3 (high)	Government ID + liveness

`assuranceLevel` is signed inside the attestation and checked against `minAssurance` in-circuit. `verificationMethod` in the API response is descriptive metadata only. Optionally bind subjects to W3C DIDs for portability.

---

10. Frontend lifecycle and testing

Lifecycle: `idle → uploading → analyzing → (rejected | review | attested) → proving → submitting → confirmed | error`. Derive the subject and sign the ownership challenge locally; drive `proving`/`submitting` from Midnight.js callbacks; show "Verified on Midnight" only after finalization **and** indexed `verifiedUntil[subject]`; no hard-coded addresses/timers/hashes; disclose data destination and mode before upload.

Unit/property: digest parity; policy tables incl. integer money boundaries and rounding; magic-byte/size/quota/auth; subject-ownership proof (accept valid, reject forged); idempotency same-body replay vs different-body 422; prompt-injection corpus.

Compact integration (compiled, real proofs), per profile:

Test	Expected
Happy path	Subject registered; nullifier consumed
Tamper any signed field (incl. assurance, amount)	Abort
Reuse valid signature + modified witness fields	Abort
Wrong key / rogue oracle	Abort
Wrong subject	Abort
Caller doesn't own claimed subject (pre-sign)	403, never signed
Assurance below minimum	Abort
Expired by chain time	Abort **[depends on SPIKE-BLOCKER]**
Replay	Second submission aborts
Revoked-then-resurrect	Pre-revocation attestation aborts
Revocation epoch backward	Aborts
Backward status overwrite	Aborts
Crash after sign, before confirm	Document not re-earnable (ISSUED durable)
eligible=false	Abort
Key rotation / revocation non-admin	Rejected
Public-state leakage	Only subject, statusValidUntil, nullifier, revokedAt
Real settlement	Finalized tx on local devnet, not mocked
Under-constrained scan	No unconstrained witness changes outcome

Golden vectors per profile, in CI against TS signer and compiled contract.

---

11. Observability

Metrics: `http_requests_total{endpoint,method,status}`, `http_request_duration_seconds`, `zkproofs_generated_total`, `zkproof_generation_time_seconds`, `zkproofs_failed_total`, `onchain_submission_latency_seconds`, `documents_verified_total`, `duplicate_attempts_total`, `subject_ownership_failures_total`. Structured logs, correlation-id-linked, **no PII/content/secrets**; alert on failure-rate, duplicate-attempt, and ownership-failure spikes.

---

12. Tooling, CI, operations

- **Pinned stack (Profile A, exact, with checksums in CI — M3):** compiler **0.31.1**, compact-runtime **0.16.0**, on-chain runtime **3.0.0**, Midnight.js **4.1.1**, proof server **8.1.0**, Node 22+. Install the exact patch versions and verify

checksums; do not use floating `compact update` in release builds. Profile B: 0.35.x `--feature-zkir-v3`, runtime 0.20.0 (devnet only).

- **CI gates (all mandatory):** compiler-version gate (reject 0.31.0, require >= 0.31.1, pin exact 0.31.1 + checksum); typecheck/lint; unit tests; golden vectors; Compact compile (both profiles); **expiry-assertion-present gate (build fails if commented)**; subject-ownership tests; dependency + secret scan; end-to-end transaction test; legal/privacy review before institutional use.
- **Secrets:** `GEMINI_API_KEY` and signer creds server-only; production signing in KMS/HSM/TEE with access logging and a tested `rotateOracleKey` runbook.
- Pin `GEMINI_MODEL`; the independent cross-check uses a *distinct* model/provider; re-run adversarial eval on any change.
- **Incident runbooks:** oracle-key compromise (rotate on-chain, revoke affected subjects with a monotonic epoch), provider breach, extraction false pos/neg.

Env vars: `GEMINI_API_KEY`, `GEMINI_MODEL`, `GEMINI_CROSSCHECK_MODEL`, `ORACLE_KEY_REF`, `CIPHERCOMPLY_POLICY_ID`, `CIPHERCOMPLY_DOMAIN`, `CIPHERCOMPLY_MIN_ASSURANCE`, `MIDNIGHT_NETWORK`, `MIDNIGHT_PROFILE`.

Repo layout: `contracts/ciphercomply.schnorr.compact`, `contracts/ciphercomply.ed25519.compact`, `contracts/managed/`, `lib/{oracle-signer,policy-engine,extraction-schema,document-registry,idempotency,subject-ownership}.ts`, `app/api/v1/{verify,status}/route.ts`, `test/...`, `compose.local.yml`, `run-devnet.sh`, `THREAT-MODEL.md`, `PRIVACY-BOUNDARY.md`, `SECURITY.md`, `.env.example`.

---

13. Roadmap and required order

Per the v4 audit's recommended sequence:

1. Resolve exact compiler/security version and pin it (**done on paper; confirm in spike**).
2. Bind authenticated identity to the oracle subject (**5.6 — implement**).
3. Implement and prove chain-time expiry (**[SPIKE-BLOCKER]**).
4. Harden revocation and registry transactions (**5.2, 7.1 — implement**).
5. Compile real contracts (both profiles).
6. Run adversarial/security tests.
7. Independent security audit.
8. Preprod on Profile A → Mainnet on Profile A → migrate to Profile B when ledger 9 ships.

Gating: **Development** now; **Preprod** only after steps 2–6; **Mainnet** only after step 7.

---

14. Validation spike (3–5 days)

1. Compile both profiles on the exact pinned stack. Confirm `persistentHash` over the struct, `jubjubSchnorrVerify` (A), `ed25519Verify<32>` (B), the Uint cast, and — the hard blocker — the **ledger-8 chain-time primitive** for `expiresAt` and the revocation bound. Exit: both compile; dummy proof verifies; expiry assertion active.
2. Gemini end-to-end on samples; confirm model output only feeds the deterministic policy; confirm inline/Files limits and MIME; validate integer-money annualization.
3. Stub `/v1/verify`; confirm 400/403/409/413/415/422 behavior, subject-ownership rejection, and durable-registry crash recovery.
4. Confirm cross-user and assurance-below-minimum fail in-circuit; confirm a non-owner is rejected before signing.
5. Observability up; no PII in logs; spikes alert.

---

15. Verified facts and residual open items

Verified (Midnight official docs + compiler-validated sources, 2026):

- All public Midnight environments run **ledger 8**; stack is compiler **0.31.1** / compact-runtime **0.16.0** / on-chain runtime **3.0.0** / Midnight.js **4.1.1** / proof server **8.1.0** (compatibility matrix).
- 0.34/0.35 target ledger 9, cannot deploy publicly.
- 0.35.0 adds `ed25519Verify<#n>` (RFC 8032, sha512 challenge, rejects identity), `sha512`, P256/Curve25519 behind `--feature-zkir-v3`.
- `jubjubSchnorrVerify` exists and rejects the identity public key.
- **Compiler 0.31.0** has a soundness bug (silently dropped range constraint), source-review-undetectable; **0.31.1 is the fix** and the network-supported pin. v3's pin was correct; the v3 audit's "0.31.1 bug" was off by one patch.
- `Uint<32> + 1` widens; cast back. Key registries need deterministic keys.
- Subject-derivation / `persistentCommit` / witness / `disclose()` patterns from the official build guide. `kernel.caller()` is `none()` for top-level user calls off-chain.

Open items — compile-dependent or process (cannot be closed on paper):

1. **[SPIKE-BLOCKER]** ledger-8 chain-time primitive for `expiresAt` and the revocation bound. Resolve before any deploy; without it Profile A has no freshness guarantee.
2. Exact call signatures of `persistentHash` over a struct, `jubjubSchnorrVerify`, `ed25519Verify<32>` against the pinned compilers.
3. Exact Profile A Schnorr signing primitive matching the verifier (H5 residual).
4. Current Gemini inline/Files size limits and accepted MIME types.
5. Independent security audit of the compiled circuits (process; cannot be self-performed).

Primary residual design risk (inherent, not a bug): the oracle/document trust root. A deceived or compromised extraction yields a valid proof of invalid data. ZK cannot address this; mitigations are verifiable evidence sources, oracle governance, independent cross-checks, and human review. This remains an explicit, accepted trust assumption.

Freeze statement: every item resolvable by document revision is resolved. The remaining five are execution and audit items. This specification is frozen; the next artifact is compiled code plus spike results, not a v6 document.

Appendix A: Full issue register (all rounds)

ID	Issue	Severity	Status
C1	Unbound digest	Blocker	Fixed v3
C2	Policy not bound	Critical	Fixed v3
C3	Subject binding not real (redemption)	Critical	Fixed v3
C4	Expiry not enforced	Critical	Gated v4; active-assertion CI gate
C5	Sealed key vs rotation	Critical	Fixed v3
C6	Document authenticity/uniqueness	Blocker	Mitigated v3; durable registry v5
C7	Open signing oracle	Blocker	Fixed v3
H-DEPLOY	Ed25519 not public	Blocker	Dual-profile v3
V3-1..5	Commented expiry, compiler, revocation resurrection, monotonic status, signed assurance	Crit/High	Fixed v4
V4-C2	Identity→subject not bound at signing	Critical	**Fixed v5 (5.6)**

ID	Issue	Severity	Status
V4-H1	Admin-controlled revocation time	High	**Fixed v5 (monotonic + chain-time)**
V4-H2	Registry not crash-safe	High	**Fixed v5 (durable state machine)**
V4-H3	Idempotency body not bound	High	**Fixed v5**
V4-H4	Float money	High	**Fixed v5 (scaled integers)**
V4-H5	Schnorr signer unspecified	High	**Specified v5; exact verifier call spike-dependent**
V4-M1	verificationMethod unsigned	Medium	**Fixed v5 (classified non-authoritative)**
V4-M2	“Independent” undefined	Medium	**Fixed v5 (distinct model/provider)**
V4-M3	Pinning not exact	Medium	**Fixed v5 (exact + checksums)**
V4-C3	Compiler claim unsourced	Process	**Fixed v5 (cited inline)**

Appendix B: Rejected recommendations, with reasons

- E2E encryption with server-side AI — contradictory; use TEE/on-device.
- On-chain document-hash registry — leaks a linkable hash; off-chain durable registry + on-chain nullifiers instead.
- Circom/Noir, Circomspect/Ecne, Solidity mapping — wrong stack; principles adopted, tools not.
- Fix oracle collusion in-circuit — impossible; ZK can’t prove the signer honest; governance + cross-checks instead.
- “0.31.1 has a Uint soundness bug” — inaccurate; the bug is 0.31.0 and 0.31.1 is the patched, network-supported version.